

A Fully Reconfigurable and Programmable VLIW Architecture for Nonlinear Function Acceleration

Ke Hu, Zikai Zhao, Yuxuan Qiao, Yifan Song, Jun Liu, Lei Xu, Wenhao Cai, Xiankui Xiong, Fan Yang *Member, IEEE*

Abstract—Nonlinear functions have become a major performance bottleneck in neural network inference due to diverse computational patterns and continued evolution. Existing solutions face a trade-off between efficiency and flexibility: fixed-function hardware is efficient but difficult to extend, whereas more general-purpose execution substrates remain inefficient for diverse nonlinear functions. We present FRV-SFU-VLIW, a fully reconfigurable and programmable Very Long Instruction Word (VLIW) architecture for accelerating nonlinear functions expressible as directed acyclic graphs (DAGs) over a configurable set of basic arithmetic, reduction, load/store, and polynomial-approximated special-function primitives. A compiler decomposes each function into a dependency-aware DAG. Special-function operations are executed by a lightweight Special Function Unit (SFU) module using adaptive non-uniform segmentation with second-order approximation, while arithmetic operations are supported by a Reconfigurable Function Unit (RFU)-based vector datapath under the Function Unit Virtualization (FUV) mechanism. By eliminating the rigid boundary between vector arithmetic and special-function resources, FUV enables fully reconfigurable allocation of vector-side compute resources between the two execution paths, allowing it to adapt to workload demands. To exploit this flexibility, a Static Multi-Mode Scheduler performs cycle-accurate scheduling across multiple FUV modes and generates VLIW instruction streams for runtime reconfiguration. Implemented with TSMC 28 nm CMOS technology, FRV-SFU-VLIW achieves up to $35.2\times$ cycle reduction and a $5.7\times$ reduction in technology-normalized area over SwiftTron nonlinear modules, an indicative cross-platform $2.0\times$ – $25.0\times$ reduction in cycles per element over FPGA-based general nonlinear accelerators, and $15.9\times$ – $26.3\times$ energy efficiency gains over a high-end GPU for standalone nonlinear functions, while maintaining negligible end-to-end accuracy loss.

Index Terms—Neural Network Acceleration, VLIW Architecture, Reconfigurable Architecture, Nonlinear Function Execution, Hardware-Software Co-design, Static Scheduling.

I. INTRODUCTION

NEURAL networks, from Convolutional Neural Networks (CNNs) [1] to Transformer architectures [2], have reshaped computing workloads across perception, language understanding, recommendation, and scientific applications. While CNNs mainly combine dense linear algebra with pointwise activations such as GELU [3], ReLU [4], and Swish [5], Transformers further place numerically sensitive global nonlinear functions such as Softmax [6] and Layer Normalization (LayerNorm) [7] on the critical path of almost every layer.

Extensive research has accelerated General Matrix Multiplication (GEMM) through sparsification, in- and near-memory

computing, and dataflow optimization [8]–[10]. As linear-kernel throughput improves, Amdahl’s Law [11] makes nonlinear stages increasingly important in end-to-end latency and energy consumption. Nonlinear functions are particularly significant in batch-1 and streaming inference: Softmax accounts for over 30% of Bidirectional Encoder Representations from Transformers (BERT) runtime [6] on the Volta GPU and approximately 30%–70% of GPT-3 XL runtime under different GEMM optimizations [12], while LayerNorm contributes approximately 16%–33% of GPT-2 latency [13]. In an FPGA-based evaluation, LayerNorm, Softmax, and GELU jointly account for 11.6% of BERT latency and 25.4% of DeiT-Small latency [14]. Since dedicated Softmax and LayerNorm units can consume over 50% of accelerator area [15], programmable accelerator support for multiple nonlinear functions is valuable across diverse models and inference settings.

Nonlinear computation differs substantially from GEMM. First, nonlinear functions combine heterogeneous operation types, including element-wise arithmetic, global reductions, special-function evaluation, and frequent data movement, which do not map efficiently onto regular matrix-processing datapaths. Second, different nonlinear functions exhibit drastically different computational structures and arithmetic intensities. As a result, hardware provisioned for one nonlinear function can be inefficient for another. As nonlinear functions continue to evolve, rigid function-specific hardware also becomes increasingly difficult to sustain.

Existing approaches therefore face a trade-off between efficiency and flexibility. Fixed-function accelerators (e.g., SwiftTron [16]) are efficient but constrained by hardwired templates and data formats. Programmable nonlinear accelerators such as NPE [17], as well as GPUs, provide greater flexibility but often achieve suboptimal throughput or energy efficiency. Conventional Very Long Instruction Word (VLIW) architectures expose Instruction-Level Parallelism (ILP), but their statically partitioned arithmetic and special-function resources can remain poorly utilized across heterogeneous nonlinear workloads. Nevertheless, the static dependencies and predictable primitive latencies of nonlinear DAGs make compiler-scheduled VLIW execution attractive if this resource imbalance can be addressed.

To bridge the gap between algorithmic evolution and hardware rigidity, we propose FRV-SFU-VLIW, a highly programmable VLIW architecture for accelerating a broad class of nonlinear functions. Instead of implementing each function as a fixed monolithic operator, the compiler decomposes it into basic arithmetic, special-function, load, and store primitives and constructs a dependency-aware directed acyclic graph (DAG) for execution.

Ke Hu, Zikai Zhao, Yuxuan Qiao, Yifan Song and Fan Yang are with the State Key Laboratory of ASIC and System, College of Integrated Circuits & Micro-Nano Electronics, Fudan University, Shanghai, China 200433.

Jun Liu, Lei Xu, Wenhao Cai and Xiankui Xiong are with the State Key Laboratory of Mobile Network and Mobile Multimedia Technology, ZTE Corporation, Shenzhen, China 518057.

The hardware combines an RFU-based reconfigurable arithmetic datapath with a lightweight Special Function Unit (SFU). The SFU employs adaptive non-uniform segmentation and second-order approximation to provide an efficient accuracy–area trade-off. Since vector special-function evaluation is reduced to quadratic polynomial arithmetic, the proposed low-overhead Function Unit Virtualization (FUV) mechanism can temporarily reuse Reconfigurable Function Units (RFUs) as the quadratic-computation backend, adapting vector-side resources to workload demands. To exploit the available FUV configurations, we develop a Static Multi-Mode Scheduler. Given the compiler-generated workload DAG, it performs cycle-accurate, resource-aware scheduling under each candidate mode, selects the minimum-latency schedule, and emits the corresponding VLIW instruction stream for runtime reconfiguration.

FRV-SFU-VLIW is an architectural framework whose primitive set can be configured according to application requirements. A DAG provides a general computation-graph representation for nonlinear functions composed of a finite set of operations connected by acyclic data dependencies. A function is supported when all nodes in its DAG can be mapped to the primitives instantiated in a given implementation. The prototype in this paper implements a practical subset of these primitives: composite functions built from existing primitives require only DAG construction and recompilation, whereas introducing a new elementary primitive may require corresponding ISA and hardware extensions.

Implemented with TSMC 28 nm CMOS technology, the prototype achieves $34.7\times$ – $35.2\times$ fewer cycles per vector and a $5.7\times$ reduction in technology-normalized area compared with the nonlinear modules of ASIC accelerator SwiftTron [16]. In an indicative cross-platform comparison with FPGA-based general nonlinear accelerators [14], [17], it delivers $2.0\times$ – $25.0\times$ fewer cycles per element. Against highly optimized fused kernels on an NVIDIA RTX 4090 GPU, our prototype achieves $15.9\times$ – $26.3\times$ improvements in energy efficiency for standalone nonlinear functions. The key contributions are:

- We propose FRV-SFU-VLIW, a fully reconfigurable and programmable VLIW architecture for nonlinear functions expressible as DAGs over a configurable set of arithmetic, reduction, special-function, and load/store primitives. Its reconfigurability enables runtime allocation of vector-side RFU resources between arithmetic and special-function execution.
- We design a lightweight SFU module and an RFU-based vector datapath. The SFU module employs adaptive non-uniform segmentation with second-order approximation, while low-overhead FUV realizes workload-adaptive RFU allocation without replicating complete quadratic-computation datapaths in the SFU.
- We develop a compiler framework that decomposes nonlinear functions into fine-grained primitives and constructs dependency-aware workload DAGs. Its Static Multi-Mode Scheduler then performs cycle-accurate, resource-aware scheduling under each FUV mode, selects the minimum-latency schedule, and emits the corresponding VLIW instruction stream.

The remainder of this paper is organized as follows. Section II reviews the background and related work. Section III

presents the FRV-SFU-VLIW architecture and FUV mechanism. Section IV introduces the compiler and scheduling framework. Section V details hardware and algorithmic implementation techniques. Section VI reports experimental results and analysis. Finally, Section VII concludes this paper.

II. BACKGROUND

Nonlinear functions are a major bottleneck in modern Artificial Intelligence (AI) workloads because they combine elementwise transformations, global reductions, and numerically sensitive special-function evaluations that are difficult to support efficiently with fixed-function or purely vector-based architectures. Transformers are representative due to their widespread adoption and their heavy reliance on nonlinear functions, although these challenges also arise in broader AI and scientific computing applications [18]–[21].

A. Nonlinear Functions in Transformers

1) *Softmax*: Softmax is a key operation in attention mechanisms, transforming similarity scores into a probability distribution:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}. \quad (1)$$

To improve numerical stability, it is typically reformulated as:

$$\text{Softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^N e^{x_j - \max(x)}}. \quad (2)$$

From a hardware perspective, Softmax forms a strictly ordered chain of maximum reduction, exponential evaluation, denominator accumulation, reciprocal, and normalization.

2) *LayerNorm*: LayerNorm normalizes inputs across the feature dimension:

$$\text{LayerNorm}(x_i) = \frac{x_i - \mu}{\sigma} \cdot \gamma + \beta, \quad (3)$$

where

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad (4)$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}. \quad (5)$$

LayerNorm similarly consists of mean reduction, variance computation, square-root and reciprocal evaluation, and elementwise normalization. In both functions, reduction, elementwise, and special-function stages form strongly ordered dependency chains, where long-latency operations block downstream execution and limit cross-stage parallelism and pipeline utilization.

3) *GELU and Activation Functions*: GELU is widely used in Transformer feedforward networks and is defined as:

$$\text{GELU}(x) = x \Phi(x), \quad (6)$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt. \quad (7)$$

GELU is an elementwise operation without reductions, but its special-function approximation requires balancing hardware cost and approximation accuracy.

B. Related Works

Existing nonlinear accelerators fall into three groups: general-purpose activation function implementations, operator-specific co-designs, and reusable nonlinear acceleration substrates.

1) General-purpose activation function implementations:

Prior works implement activation functions such as Tanh and Sigmoid in hardware. CORDIC-based designs [22] reuse iterative computation units across multiple functions, while piecewise approximation methods, including polynomial and linear interpolation approaches [23]–[26], reduce computational complexity. Flex-SFU [27] employs non-uniform segmentation for improved accuracy–area trade-offs, and stochastic computing methods [28] offer alternative implementations.

However, these approaches primarily target elementwise functions and do not extend naturally to operations involving global reductions, such as Softmax and LayerNorm. As a result, they cannot provide a unified solution for the diverse nonlinear patterns in modern AI workloads.

2) Operator-specific co-designs:

Some works focus on optimizing specific nonlinear operators. For Softmax, techniques such as integer-based approximations [29] and base-2 exponential reformulations [6] reduce computational cost. Other designs integrate dedicated hardware units for Softmax, LayerNorm, and activation functions [15], [16], [30], [31].

While these approaches achieve high efficiency for targeted operators, they rely on fixed-function hardware, limiting their flexibility and scalability. Extending such designs to support new nonlinear functions or different precision formats typically requires significant redesign effort.

3) Reusable nonlinear acceleration substrates:

Recent works explore reusable programmable substrates for broader nonlinear-operator support. MEGA.mini [32] employs shared quadratic-polynomial processing elements to support multiple activation functions, but it does not extend to reduction-based operators such as Softmax. Qin et al. [33] propose a tri-directional systolic workflow with support for activation and Softmax, yet LayerNorm is not covered. Kim et al. [34] design an FPGA-based accelerator that supports Softmax and LayerNorm using piecewise linear approximations and configurable processing elements; however, the design remains largely fixed-function and lacks extensibility to new nonlinear operators. The Tandem processor [35] composes nonlinear operators from vector arithmetic primitives with an emphasis on data movement optimization, but it does not address the design of efficient SFUs, which are critical for accurate nonlinear evaluation. NPE [17] introduces a unified piecewise–polynomial approximation framework within an FPGA overlay, providing programmability across multiple nonlinear functions. However, its evaluation is limited to fixed-point implementations, and instruction scheduling relies on microcoded control, which may underutilize instruction-level parallelism. TATAA [14] adopts a dual-mode architecture that switches between INT8 systolic execution and BF16 SIMD vector processing to improve workload coverage. Nevertheless, the design is tightly coupled to a fixed precision configuration, and extending it to additional formats requires substantial co-design effort.

Overall, while these designs improve flexibility compared to fixed-function accelerators, they still exhibit key limitations, in-

cluding incomplete operator coverage (especially for reduction-based nonlinearities), a lack of efficient support for special-function evaluation, and limited scalability across different precision and workload requirements. These observations indicate that a more effective nonlinear accelerator should support not only broad programmability, but also flexible sharing and reuse of computing resources across different execution paths to better balance efficiency and flexibility.

III. FRV-SFU-VLIW ARCHITECTURE

This section presents the hardware architecture of FRV-SFU-VLIW. The design combines lightweight SFUs with an RFU-based arithmetic backbone to support nonlinear workloads expressible using the implemented primitives. Through FUV, RFUs can be dynamically allocated between general arithmetic and the quadratic-computation backend of vector special functions according to workload demands. Section III-A introduces the overall architecture, Section III-B describes the lightweight SFU organization, and Section III-C presents the RFU-based arithmetic backbone and FUV mechanism.

A. Architecture Overview and Design Rationale

FRV-SFU-VLIW adopts a customized VLIW execution model to expose fine-grained ILP to the compiler. The design offloads dependency analysis and operation ordering to a static scheduler, enabling deterministic latency and simpler hardware control. To support large workloads, the system follows a block-wise execution strategy. Target nonlinear kernels are partitioned into manageable tiles and sequentially streamed from external memory (e.g., High Bandwidth Memory (HBM)) or a GEMM engine into the Ping-Pong Buffer. A tile is launched only after its input has been fully transferred into one Ping-Pong Buffer and the preceding tile has completed. The core then fetches one VLIW instruction word per cycle until an *End-of-Program (EOP)* instruction is decoded, while external transfers are controlled independently; thus, an HBM stall delays only the next tile launch without interrupting or rescheduling the current static instruction stream.

As shown in Figure 1, FRV-SFU-VLIW consists of a software front-end and a hardware core. Given a workload expressible as a DAG over the supported primitives and the hardware parameters, the front-end performs the three steps shown in Figure 1—DAG construction, FUV-mode analysis, and Static Multi-Mode Scheduling—and emits the resulting VLIW instruction stream to the hardware core. Within the hardware core, the storage subsystem, highlighted in green, consists of the multi-banked Scalar General Purpose Registers (SGPRs) and Vector General Purpose Registers (VGPRs), which store scalar and vector operands, respectively. The computation subsystem, highlighted in yellow, is composed of three main modules: the Scalar module, the Vector module, and the SFU module. The Scalar module is built around the Baseline Function Unit (BFU) for scalar arithmetic operations. The Vector module is built around an RFU array of size N , together with dedicated reduction support. The dedicated reduction datapaths perform hierarchical accumulation internally, avoiding repeated VGPR/SGPR accesses to partial reduction results. Each RFU serves as a basic reconfigurable arithmetic unit for vector computation, and multiple RFUs collectively form the main vector

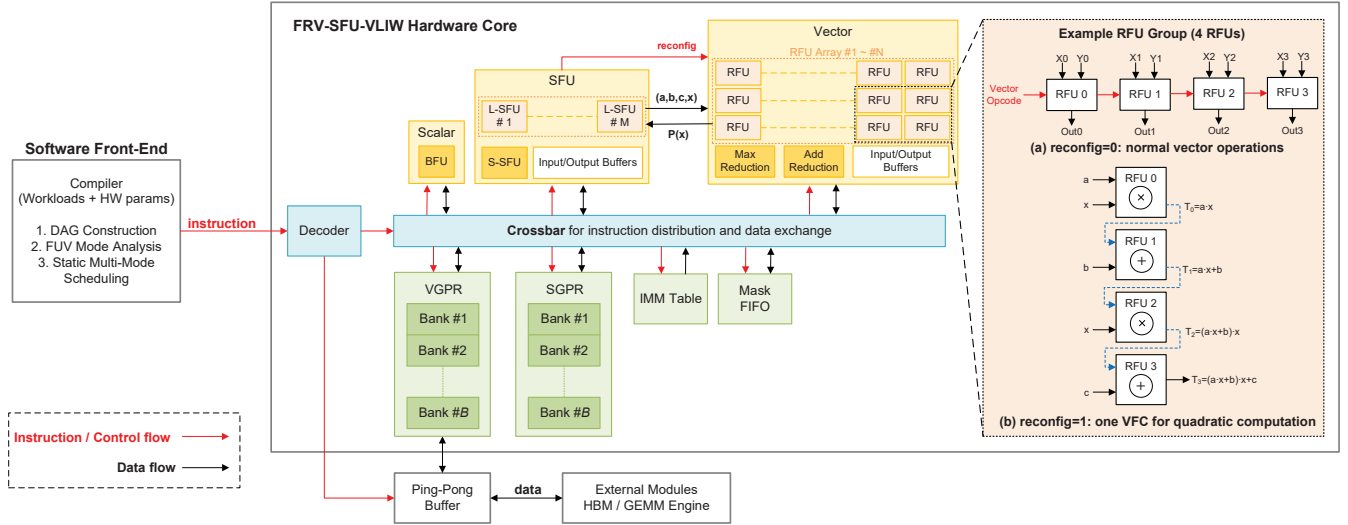


Fig. 1. Overall architecture of FRV-SFU-VLIW. The software front-end generates statically scheduled VLIW instructions, while the hardware core integrates SGPR/VGPR storage, Scalar/Vector/SFU computation modules, a shared crossbar, and a decoder for nonlinear function execution.

execution engine. The SFU module is responsible for special-function execution and consists of one Scalar SFU (S-SFU) and multiple Lightweight SFU Lanes (L-SFUs). A decoder parses each VLIW instruction word, while the shared crossbar distributes the decoded control fields to the corresponding computation modules and provides deterministic-latency data routing between register-bank and computation-module ports. For vector data, the lanes maintain fixed one-to-one correspondence without cross-lane permutation; thus, with a fixed number of module ports, the logical data-routing complexity scales as $O(N)$, although monolithic widening remains limited by increasing wire length, capacitive load, and routing congestion. As future work, we will explore wider implementations using fixed-width slices with local register banks, computation resources, and decoder/crossbar logic, connected by a narrow hierarchical network for instruction/control distribution and cross-slice reduction aggregation.

The VLIW instruction word is organized into five functional slots and one control slot: LOAD, STORE, SCALAR, VECTOR, SFU, and CONTROL. LOAD and STORE slots manage bidirectional data transfer between the Ping-Pong Buffer and VGPRs; format conversion can be incorporated at this boundary when needed. The SCALAR slot controls the BFU, the VECTOR slot drives the RFU array and reduction datapaths, and the SFU slot coordinates special-function execution through the S-SFU and L-SFUs. This organization reflects the software–hardware co-design rationale of FRV-SFU-VLIW. The software front-end decomposes nonlinear workloads into fine-grained primitives, exploits ILP through static resource-aware scheduling, and encodes cycle-level control into VLIW instructions, thereby reducing dynamic hardware-control overhead. The hardware core provides parallel, shared, and reconfigurable execution resources to efficiently execute the generated instruction streams across different workloads.

B. Lightweight SFU Organization

The SFU subsystem is dedicated to processing complex special-function operations, such as e^x , $\ln x$, and GELU, which

are computationally expensive for conventional arithmetic units. To achieve a balanced trade-off among numerical accuracy, silicon area, and execution latency, FRV-SFU-VLIW adopts a piecewise quadratic polynomial approximation, $P(x) = ax^2 + bx + c$, together with non-uniform segmentation. Compared with iterative CORDIC-style implementations or large interpolation tables, this approach provides a more suitable foundation for a high-throughput architecture with deterministic execution latency and low storage overhead. Based on quadratic polynomial approximation, the SFU module is organized into one S-SFU and multiple L-SFUs, thereby preserving generality while reducing the fixed hardware cost of special-function support.

As illustrated in Figure 2, the SFU operator datapath follows a fully pipelined flow consisting of format conversion, pre-processing, coefficient lookup, quadratic computation, and post-processing. To maintain numerical fidelity, input data is first converted from the external interface format to an internal higher-precision representation. For example, our prototype upcasts external FP16 data to a custom FP19 (E5M13) format to satisfy function-specific error tolerances on the order of 10^{-4} – 10^{-3} . Developers can adjust the internal data format according to specific application requirements.

Following format conversion, the datapath performs operation-specific pre-processing (i.e., range reduction) based on the operation type (Otype) to map x' into a minimal table-searchable domain x . Concurrently, metadata required for subsequent post-processing is extracted and propagated down the pipeline to ensure cycle-level synchronization. Architecturally, a B_{op} -bit Otype field can support up to $2^{B_{op}}$ distinct SFU operations. In our implementation, we allocate a 3-bit Otype to accommodate seven commonly used operations: exponential (EXP), square root (SQRT), reciprocal (INV), natural logarithm (LN), GELU, Tanh, and Sigmoid. In the non-uniform segmented lookup stage (highlighted in purple in Figure 2), the hardware executes a fast search to determine the segment index and fetches the pre-computed coefficients (a, b, c) from the local LUT associated with the specified Otype.

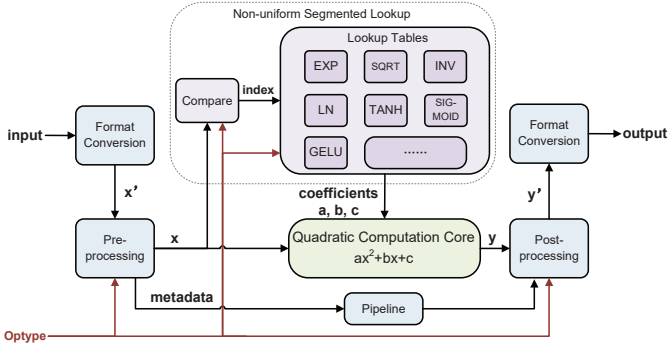


Fig. 2. Structure of the proposed SFU datapath for polynomial-based special-function execution.

The mapped input x and the retrieved coefficients are then routed to the quadratic computation core (highlighted in green in Figure 2). For scalar operations, the S-SFU contains a local quadratic computation path and directly completes polynomial evaluation. For vector operations, however, the L-SFUs do not include dedicated quadratic computation hardware; instead, they cooperate with reusable RFU resources in the Vector module to complete the polynomial evaluation. Regardless of the operator type, both execution paths are designed to share an identical pipeline depth, thereby ensuring cycle-level alignment without incurring additional latency overhead. Finally, the post-processing stage reconstructs the output and converts it back to the target format.

Supported by this lightweight SFU organization, the architecture retains the flexibility of polynomial-based special-function execution while reducing the fixed hardware overhead by removing fully replicated local quadratic datapaths from vector special-function lanes. The generation of non-uniform segments and polynomial coefficients is performed offline by a software-driven procedure, which will be introduced in Section V.

C. RFU-Based Vector Module and FUV Mechanism

Conventional VLIW architectures usually bind functional resources to fixed execution paths. However, nonlinear workloads often exhibit highly unbalanced demands across arithmetic and special-function stages. For SFU-intensive kernels such as GELU and Softmax, the special-function path may become the throughput bottleneck, while a large fraction of vector arithmetic resources remains underutilized. This mismatch degrades performance and leads to substantial “dark silicon” inefficiency.

FRV-SFU-VLIW addresses this issue through an RFU-based Vector module under a unified FUV mechanism. Since the SFU module standardizes diverse special functions into a common piecewise quadratic polynomial approximation, the core computation can be reduced to arithmetic operations on polynomial coefficients and pre-processed inputs. Therefore, the Vector module serves not only as the main engine for normal vector arithmetic, but also as a reusable computation backend for vector special-function execution. As shown in Figure 1, the Vector module consists of an RFU array together with dedicated reduction support. The RFU array is organized into local RFU groups, each of which can either remain in normal vector mode or be reconfigured into a Virtual Function Cluster (VFC) for quadratic polynomial evaluation.

The proposed hardware supports multiple RFU virtualization granularities for vector special-function execution. The corresponding compiler-visible FUV modes determine how many RFU groups in the Vector module are allocated to VFC-based computation, while the detailed mode definition and selection strategy are deferred to Section IV. At runtime, each four-RFU group is controlled by a 1-bit *reconfig* state. When *reconfig*=0, the four RFUs perform normal vector arithmetic operations independently; when *reconfig*=1, they are reorganized into one VFC for quadratic computation, as illustrated in Figure 1.

The virtualization process is tightly coordinated between the SFU module and the Vector module. For vector special-function operations, the L-SFUs first execute the frontend stages, including format conversion, pre-processing, and non-uniform segmented coefficient lookup. After the lookup stage completes, the SFU controller issues a reconfiguration request and asserts the corresponding reconfig signal for the target RFU group, while simultaneously dispatching the reduced input and polynomial coefficients (x, a, b, c) to the Vector module. Once *reconfig*=1 is enabled, the selected RFU group temporarily stops serving normal vector operations and is reconfigured into a VFC to perform quadratic polynomial evaluation. The resulting polynomial output $P(x)$ is then returned to the originating L-SFU, which completes the backend stages, including post-processing and output format conversion. After the polynomial evaluation finishes, the reconfig signal is deasserted, and the RFU group is released back to its default vector execution state.

Through this mechanism, FRV-SFU-VLIW enables RFU resources to be dynamically reused by vector special-function operations, thereby increasing vector special-function parallelism without permanently replicating dedicated quadratic computation hardware inside all L-SFUs. More importantly, such reuse is explicitly supported by the customized VLIW instruction set and orchestrated by compiler-generated instruction streams.

IV. COMPILER FRAMEWORK

To fully leverage the architectural reconfigurability of FRV-SFU-VLIW, we develop a compiler framework that bridges diverse nonlinear workloads and the hardware substrate. The compilation flow consists of three stages: workload decomposition and DAG construction, FUV mode space analysis, and Static Multi-Mode Scheduling. In addition, workload tiling reduces instruction and buffer footprints for large workloads.

A. Workload Decomposition and DAG Construction

The first stage of the compiler is to translate the target workload into a hardware-executable intermediate representation, ensuring programmability for diverse and evolving nonlinear functions expressible through the instantiated primitives.

Unlike fixed-function accelerators that treat nonlinear functions as monolithic blocks, our compiler decomposes each workload into a sequence of fine-grained, operation-level primitives. These primitives include element-wise arithmetic (e.g., *ADD*, *MUL*), collective reductions (e.g., *REDUCE_MAX*, *REDUCE_ADD*), and special-function operations (e.g., *EXP*, *GELU*). These operations are mapped into nodes of a DAG, capturing the implicit data flow and dependencies inherent in the workload. As examples, Figures 3 and 4 illustrate the

Node	Operation	Description	Children
0	NOP	root	1
1	LOAD	LOAD(x)	2
2	REDUCE_MAX	m = REDUCE_MAX(x)	3
3	SUB	y = x - m	4
4	EXP	z = EXP(y)	5, 7
5	REDUCE_ADD	s = REDUCE_ADD(z)	6
6	INV	r = 1 / s	7
7	MUL	t = z * r	8
8	STORE	STORE(t)	None

Fig. 3. Operation breakdown of Softmax on an unsegmented vector of length N .

Node	Operation	Description	Children
0	NOP	root	4, 5, 6
1	NOP	$\sqrt{2}$	15
2	NOP	ϵ	13
3	NOP	1/d	8, 12
4	LOAD	LOAD(γ)	18
5	LOAD	LOAD(β)	19
6	LOAD	LOAD(x)	7, 9
7	REDUCE_ADD	a = REDUCE_ADD(x)	8
8	MUL	$\mu = a * (1/d)$	9
9	SUB	b = x - μ	10
10	MUL	sq = b * b	11
11	REDUCE_ADD	c = REDUCE_ADD(sq)	12
12	MUL	e = c * (1/d)	13
13	ADD	f = e + ϵ	14
14	SQRT	g = SQRT(f)	15
15	MUL (mask)	$\sigma = g * \sqrt{2}$	16
16	INV	1/ σ	17
17	MUL	h = b * (1/ σ)	18
18	MUL	i = h * γ	19
19	ADD	j = i + β	20
20	STORE	STORE(j)	None

Fig. 4. Operation breakdown of LayerNorm on an unsegmented vector of length N . Note that Node 14 approximates $\sqrt{M}$ over the reduced domain using the SFU module. Node 15 then conditionally multiplies the result by $\sqrt{2}$ for an odd input exponent [36], which is executed by a general-purpose arithmetic unit to reduce the complexity of the SFU datapath.

operation breakdown and DAG construction for Softmax and LayerNorm functions, respectively. As an extension example, LogSumExp is rewritten as

$$m = \max_i x_i, \quad (8)$$

$$y = m + \ln \sum_i e^{x_i - m}, \quad (9)$$

and mapped to the *REDUCE_MAX*, *SUB*, *EXP*, *REDUCE_ADD*, *LN*, and *ADD* nodes. Since all required primitives are already supported, this extension only requires constructing a new DAG and recompiling it. The existing *EXP/LN* Otypes, coefficient tables, latency and resource definitions, and generic scheduling procedure are reused without ISA or hardware modifications.

For cycle-level scheduling, each DAG node encapsulates critical information, including the operation type, data type (scalar or vector), operand sources, and register allocation constraints.

This unified node structure allows the scheduler to reason about both functional correctness and architectural constraints when mapping nodes onto the BFU-based Scalar module, RFU-based Vector module, and SFU module. Furthermore, this stage allows customization of node importance metrics to guide scheduling priorities. For the repeated, strongly ordered vector DAGs evaluated in this work, nodes are numbered topologically within each vector instance and consecutively across vector instances. The negated node index therefore implements a lightweight oldest-vector-first priority that advances earlier vectors toward completion and releases their live registers sooner.

To support input vectors of arbitrary length exceeding the hardware execution width (N), the compiler uses a segmentation strategy during graph construction, dividing the workload into multiple segments. For each segment, the compiler instantiates a uniform operation sequence to process data in parallel, creating a simple, parallel structure for the bulk of the computation. For global functions, hierarchical scalar reduction nodes are inserted to aggregate partial results layer-by-layer across all segments into a single global value. Ultimately, by decoupling the workload definition from specific hardware vector-width constraints, this graph construction phase provides the flexibility to map a wide range of current and future nonlinear workloads expressible through the instantiated primitives onto FRV-SFU-VLIW regardless of their input length.

B. FUV Mode Space Analysis

In principle, the architecture could support a wide range of RFU virtualization ratios. In the prototype, the supported mode set is determined through hardware–software co-design. Based on the following pass-count analysis, we select representative virtualization ratios that capture the major workload trade-offs while preserving a regular hardware organization. The compiler then exhaustively schedules each implemented mode and selects the minimum-cycle result, without heuristic pruning within the supported mode set.

In the prototype, the standard vector length is denoted by N , which also equals the size of the RFU array. In this work, we set $N = 256$. Let M denote the number of VFCs allocated for vector special-function execution. Since each VFC consumes four RFUs, the remaining vector arithmetic capacity becomes $N - 4M$. Correspondingly, the number of passes required by the vector special-function path and the normal vector arithmetic path can be calculated as

$$P_{sfu} = \left\lceil \frac{N}{M} \right\rceil, \quad P_{vec} = \left\lceil \frac{N}{N - 4M} \right\rceil. \quad (10)$$

The above equation isolates the deterministic mode-dependent component of node latency: once issued, a vector or vector-SFU node has an intrinsic latency equal to its pass count plus a mode-independent pipeline bias. It therefore identifies the Pareto-efficient node-latency tradeoffs before scheduling, while DAG node readiness, bank conflicts, and resource contention, which depend on the actual issue order, are handled by the subsequent scheduler.

The pass-count analysis first reduces the fine-grained numerical space to a small set of structurally distinct candidate regions. Considering these regions together with the regular hardware

organization of the prototype, we instantiate three representative FUV modes and exhaustively schedule each of them:

- **Mode 0** ($M = 32$): 50% RFU virtualization, corresponding to $P_{sfu} = 8$ and $P_{vec} = 2$, suitable for workloads with relatively low vector special-function demand.
- **Mode 1** ($M = 48$): 75% RFU virtualization, corresponding to $P_{sfu} = 6$ and $P_{vec} = 4$, providing an intermediate trade-off for more balanced workloads.
- **Mode 2** ($M = 64$): 100% RFU virtualization, corresponding to $P_{sfu} = 4$, during which vector instructions are temporarily disabled, suitable for workloads dominated by vector special-function execution.

The three modes cover vector-dominant, balanced, and SFU-dominant workloads. In the evaluated sweep with $X = 32$ and $Y = 64$ –1024, LayerNorm always selects Mode 0, GELU always selects Mode 2, and Softmax selects the balanced Mode 1 in only two of the 16 cases. Therefore, the prototype retains only the hardware-regular $M = 48$ configuration to cover this narrow intermediate region. The same analysis can be reapplied to other hardware configurations.

C. Static Multi-Mode Scheduling

The Static Multi-Mode Scheduler extends resource-constrained list scheduling to the FUV architecture. Given the operation DAG $\mathcal{G}$ and candidate FUV modes $\mathcal{M}$, Algorithm 1 initializes, for each mode $m \in \mathcal{M}$, the corresponding resource limits, including the RFU groups available for normal vector execution, the VFCs available for vector special-function execution, and the other shared-resource constraints. It then invokes SCHEDULEUNDERMODE to generate a legal instruction stream $\mathcal{I}_m$ and its issue cycle count T_m , and outputs the minimum-cycle instruction stream as $\mathcal{I}_{opt}$.

Within SCHEDULEUNDERMODE, scheduling proceeds cycle by cycle. At cycle t , the scheduler constructs a *ReadyList* of unissued nodes whose predecessors have completed and whose inputs are available by t . The ready nodes are sorted by importance and examined in that order; the importance metric only determines the priority among dependency-ready nodes. A node v is issued into $Instr_t$ only if its VLIW slot and target execution unit are available, its SGPR/VGPR accesses can be legally scheduled, and the mask and live-register constraints are satisfied. Per-resource tables track register-bank read/write occupancy and execution-unit availability according to previously issued nodes and their pipeline latencies. The live-register limit is derived from the GPR capacity and bounds the number of simultaneously in-flight vectors. Once issued, v is marked as issued, its output-ready time is calculated, and the resource tables are updated. Otherwise, it is reconsidered in a later cycle, while unused slots are filled with NOP.

Scheduling under mode m continues until all nodes are issued, after which an EOP marker is appended. All conflicts are resolved at compile time using cycle-indexed resource tables, while the hardware executes one generated VLIW word per cycle without stalling; thus, each scheduled issue cycle exactly matches the hardware execution. Since the final operation is always a fixed three-cycle *STORE*, the total latency is $T_m + 3$, and the compiler selects the corresponding minimum-latency stream $\mathcal{I}_m$ as $\mathcal{I}_{opt}$.

Algorithm 1 Static Multi-Mode Scheduling for FUV-Aware VLIW Compilation

```

1: Input: Operation DAG  $\mathcal{G}$ , candidate FUV modes  $\mathcal{M}$ 
2: Output: Optimized instruction stream  $\mathcal{I}_{opt}$ 
3:  $T_{best} \leftarrow \infty$ ,  $\mathcal{I}_{opt} \leftarrow \emptyset$ 
4: for each mode  $m \in \mathcal{M}$  do
5:   Initialize mode-dependent resource limits
6:    $(\mathcal{I}_m, T_m) \leftarrow \text{SCHEDULEUNDERMODE}(\mathcal{G}, m)$ 
7:   if  $T_m < T_{best}$  then
8:      $T_{best} \leftarrow T_m$ 
9:      $\mathcal{I}_{opt} \leftarrow \mathcal{I}_m$ 
10:  end if
11: end for
12: return  $\mathcal{I}_{opt}$ 

13: function SCHEDULEUNDERMODE( $\mathcal{G}, m$ )
14:   Initialize cycle  $t \leftarrow 0$ ; mark all nodes as unissued
15:    $\mathcal{I}_{seq} \leftarrow \emptyset$ 
16:   while unissued nodes exist do
17:     Initialize  $Instr_t$  with NOP
18:     Build ReadyList from dependency-resolved nodes
19:     Sort ReadyList by importance
20:     for each node  $v$  in ReadyList do
21:       if slot and target unit are available and no
       SGPR/VGPR bank conflict occurs and mask/live-register
       constraints are satisfied then
22:         Issue  $v$  into  $Instr_t$ 
23:         Mark  $v$  as issued
24:         Update resource tables and ready time of  $v$ 
25:       end if
26:     end for
27:     Append  $Instr_t$  to  $\mathcal{I}_{seq}$ 
28:      $t \leftarrow t + 1$ 
29:   end while
30:   Insert EOP into  $\mathcal{I}_{seq}$ 
31:   return  $(\mathcal{I}_{seq}, t)$ 
32: end function

```

D. Discussion on Workload Tiling

Scheduling a large workload as a single instance would increase the instruction and buffer footprints, particularly with the wide VLIW format. We therefore partition a large batch into smaller tiles and reuse the statically generated instruction stream across regular tiles, such that the required storage is bounded by the tile size rather than the complete workload. Ping-Pong buffering further overlaps data transfer and computation across consecutive tiles.

Consider Softmax (512×512), which denotes $X=512$ independent vectors of length $Y=512$. Since vectors are independent, we tile the batch into R sub-batches, each of size X/R , and execute them sequentially. With tiling, the Ping-Pong Buffer capacity scales by $1/R$ (from $O(XY)$ to $O(XY/R)$), while the instruction footprint reduces to slightly above $1/R$ of the monolithic case due to boundary under-utilization of VLIW slots. As a result, the total runtime increases modestly relative to the monolithic execution. Table I illustrates this trade-off for Softmax (512×512), where external data transfer latency is modeled using conservative HBM estimates. By selecting

TABLE I
WORKLOAD TILING OF SOFTMAX (512×512) TRADE-OFF VERSUS TILING
FACTOR R

R	Buffer size (KB)	Instruction count	Total cycles
2	512	3662	10378
4	256	1848	8984
8	128	972	8636
16	64	518	8782
32	32	288	10296

a moderate R , designers can substantially reduce buffer and instruction memory with only a slight increase in latency, yielding a practical operating point.

V. IMPLEMENTATION DETAILS

This section presents the hardware and algorithmic implementations underlying FRV-SFU-VLIW's efficiency and numerical fidelity. We first describe an adaptive non-uniform segmentation algorithm that generates compact SFU coefficient tables under given accuracy constraints. Then, we describe the dedicated hierarchical reduction datapaths integrated within the Vector module, which are explicitly designed to eliminate the iterative latency bottlenecks inherent in global reduction operations.

A. Adaptive Non-uniform Segmentation of SFU

The accuracy of polynomial approximation is heavily contingent on the segmentation of the table-searchable domain. Uniform segmentation strategies often result in redundant storage for linear-like regions or insufficient precision for high-curvature intervals. To address this, we employ an offline *Adaptive Non-uniform Segmentation Algorithm* (Algorithm 2). Given a target function $f(x)$ and strict accuracy constraints (relative tolerance τ_{rel} and absolute tolerance τ_{abs}), the algorithm adaptively allocates denser segments to intervals with high second-order derivatives while using fewer segments for flatter regions.

As detailed in Algorithm 2, the procedure processes intervals from a work queue. For each candidate sub-segment $[x_{\text{start}}, x_{\text{end}}]$, we apply a degree-2 Remez-exchange algorithm [37] via the `RUNREMEZONSEGMENT` solver. Unlike standard least-squares fitting, this approach seeks a minimax approximation, optimizing the coefficients (a, b, c) to ensure that the maximum relative error is minimized uniformly across the interval.

Once the Remez solver converges, the resulting quadratic polynomial is validated against both the relative and absolute error tolerances. Should the approximation fail to meet these strict fidelity constraints, the interval is bisected, and the resulting sub-intervals are re-enqueued for further processing. Upon exhausting the work queue, the final set of segment boundaries and corresponding coefficient tuples is sorted and stored into the on-chip coefficient table. For a new elementary special function, this algorithm can generate the segment boundaries and coefficients after range reduced to a bounded domain.

Table II summarizes the table-searchable domains and the segment counts determined by Algorithm 2 for the seven special functions supported by the prototype. The reported errors are obtained from end-to-end evaluations with FP16 inputs and

Algorithm 2 Adaptive Non-uniform Segmentation Algorithm

Require: Function f , derivative f' , domain $[L, U]$, tolerance $\tau_{\text{rel}}, \tau_{\text{abs}}$

Ensure: Set of segments $\mathcal{S} = \{([x_{\text{start}}, x_{\text{end}}], \text{coeffs}_i)\}$

```

1: if  $L = -U$  then
2:    $\mathcal{Q} \leftarrow \{[L, 0], [0, U]\}$ 
3: else
4:    $\mathcal{Q} \leftarrow \{[L, U]\}$ 
5: end if
6:  $\mathcal{S} \leftarrow \emptyset$ ,  $\text{MAXSEGMENTS} \leftarrow 100$ 
7: while  $\mathcal{Q} \neq \emptyset$  and  $|\mathcal{S}| < \text{MAXSEGMENTS}$  do
8:    $[x_{\text{start}}, x_{\text{end}}] \leftarrow \text{POPFRONT}(\mathcal{Q})$ 
9:    $(\text{coeffs}, \text{success}) \leftarrow \text{RUNREMEZONSEGMENT}(f, f', [x_{\text{start}}, x_{\text{end}}])$ 
10:  if not success then
11:    continue
12:  end if
13:   $X_{\text{dense}} \leftarrow \text{Linspace}(x_{\text{start}}, x_{\text{end}}, 4096)$ 
14:  Let  $\text{coeffs} = (a, b, c)$ ;  $P(x) = ax^2 + bx + c$ 
15:   $\text{err}_{\text{abs}} \leftarrow \max_{x \in X_{\text{dense}}} |P(x) - f(x)|$ 
16:   $\text{err}_{\text{rel}} \leftarrow \max_{x \in X_{\text{dense}}} \left| \frac{P(x) - f(x)}{f(x)} \right|$ 
17:  if  $\text{err}_{\text{rel}} < \tau_{\text{rel}}$  and  $\text{err}_{\text{abs}} < \tau_{\text{abs}}$  then
18:     $\mathcal{S} \leftarrow \mathcal{S} \cup \{([x_{\text{start}}, x_{\text{end}}], \text{coeffs})\}$ 
19:  else
20:     $x_{\text{mid}} \leftarrow (x_{\text{start}} + x_{\text{end}})/2$ 
21:     $\text{PUSHBACK}(\mathcal{Q}, [x_{\text{start}}, x_{\text{mid}}])$ 
22:     $\text{PUSHBACK}(\mathcal{Q}, [x_{\text{mid}}, x_{\text{end}}])$ 
23:  end if
24: end while
25:  $\text{SORTBYSTART}(\mathcal{S})$ 
26: return  $\mathcal{S}$ 

```

outputs and FP19 internal computation. Across all functions, the Max Relative Error Distance (Max RED) and Normalized Mean Error Distance (NMED) remain below 6.92×10^{-2} and 1.60×10^{-4} , respectively. The largest Max RED values occur near zero-valued reference outputs, where small absolute errors are relatively amplified; nevertheless, low NMED is achieved using only 4–28 segments.

Range reduction keeps the lookup domains of EXP, LN, SQRT, and INV fixed, while the outer regions of the activation functions use trivial constant or identity segments. Thus, the coefficient storage scales linearly with the segment count selected by Algorithm 2 under the target error constraints. It therefore remains unchanged for an extended range-reducible input domain and increases only marginally when the activation-function range is extended.

B. Reduction Datapaths in Vector Module

Global reduction operations, such as vector summation (`REDUCE_ADD`) and maximum search (`REDUCE_MAX`), are critical bottlenecks in Softmax and LayerNorm functions. Conventional software-based reductions on standard SIMD architectures typically require iterative pairwise operations, resulting in deep dependency chains ($O(\log N)$) and prohibitive register file read/write traffic.

To mitigate this bottleneck, the Vector module integrates dedicated hierarchical reduction trees parallel to the RFU array, en-

TABLE II
NON-UNIFORM SEGMENTATION AND ERROR ANALYSIS OF THE PROPOSED SFU DESIGN.

Function	Searchable Domain	Evaluation Range	No. of Segments	Max RED ($\times 10^{-3}$)	NMED ($\times 10^{-4}$)
EXP	[0, 1)	[-9, 9]	4	1.080	0.185
SQRT	[1, 2)	[0, 10]	4	0.906	1.600
INV	[1, 2)	[0.001, 10]	9	0.771	0.002
LN	[2, 4)	[1.1, 10]	5	2.250	1.420
GELU	[-4, 4]	[-4, 4]	25	69.200	0.421
Tanh	[-4, 4]	[-4, 4]	28	4.550	0.599
Sigmoid	[-8, 8]	[-8, 8]	21	42.100	0.794

Note: Max RED denotes the maximum relative error. NMED denotes the normalized mean error distance. Approximately one million uniformly spaced, FP16-quantized inputs are evaluated over each range.

abling collective operations to be executed as single-instruction primitives with deterministic, ultra-low latency. Specifically, to accelerate the maximum value search, the design employs a *Max-Reduction Tree* constructed as a pipelined comparator network. For a vector length of $N = 256$, this is structured as four hierarchical levels of 4-input comparators (one registered level per cycle). This dedicated path resolves the global maximum in just 4 cycles, significantly outperforming iterative comparisons. The *Max-Reduction Tree* returns only the maximum value; operators requiring index propagation or multiple-candidate selection, such as ARGMAX and TOP-K, are not implemented and fall outside the support scope of the evaluated prototype.

Complementing this is the *Add-Reduction Tree* for summation tasks. To ensure numerical stability, the datapath finds a shared maximum exponent (Cycle 1 performs FP16 decoding and signed-mantissa generation, followed by two levels of 16-input exponent search in Cycles 2–3), aligns all mantissas once (Cycle 4), accumulates them through two pipelined 16-way widened-mantissa stages (Cycles 5–6), and performs normalization (Cycle 7) and rounding only at the output (Cycle 8, including special-value selection and output registration). Therefore, it completes a 256-element floating-point summation in eight cycles without cascading regular floating-point adders.

Synthesis results indicate that instantiating these dedicated reduction trees increases the Vector module’s area by only 18.4%. However, this modest silicon overhead yields dual architectural benefits. First, it significantly reduces the contribution of global reductions to the critical execution path. Second, and more importantly, offloading these tasks from the primary vector datapath liberates the valuable generic RFUs. Consequently, these unburdened vector resources can be fully dedicated to standard element-wise arithmetic or dynamically repurposed to assist the L-SFUs via the FUV mechanism, thereby maximizing overall hardware utilization. In a matched comparison that keeps the process, frequency, vector width, register organization, and the remaining datapaths unchanged, the dedicated reduction trees achieve $3.45\times$ and $3.75\times$ speedup over reductions executed on the generic vector datapath for Softmax (512×512) and LayerNorm (512×768), respectively.

Table III reports the cycle-level latencies of this design, measured as input-to-output pipeline distances. The reported cycles reflect per-operator latencies, which remain independent of the vector length or the FUV mode.

TABLE III
PER-OPERATOR PIPELINE LATENCIES FOR ARITHMETIC AND SPECIAL-FUNCTION OPERATIONS

Operation	Cycles
Element-wise (e.g., ADD, MUL)	3
REDUCE_MAX	4
REDUCE_ADD	8
EXP	17
SQRT	12
INV	12
LN	15
GELU / SIGMOID / TANH	12

VI. EXPERIMENTS AND ANALYSIS

This section evaluates FRV-SFU-VLIW in four dimensions: end-to-end numerical accuracy, synthesis results using TSMC 28 nm CMOS technology, performance benchmarking against state-of-the-art accelerators and a high-end GPU, and a detailed quantitative analysis of the FUV mechanism.

A. Numerical Accuracy Evaluation

To evaluate the numerical accuracy of the proposed nonlinear kernel, we replace the nonlinear computation modules in representative deep learning models with our kernel implemented in PyTorch. We then measure the end-to-end accuracy (or perplexity) and compare it with the original FP32 baseline. The results are summarized in Table IV. In our prototype implementation, the system supports three numerical formats: FP16, BF16, and a custom FP19. These formats are employed in conventional vector/scalar computations, in vector/scalar computations that require an extended dynamic range, and within the quadratic polynomial computations of the SFU, respectively.

The simulation results show that the BERT model on the GLUE tasks exhibits a negligible variation in accuracy within the range of $0\% \sim +0.02\%$, while the ViT model on image classification varies by $-0.01\% \sim +0.02\%$. For the GPT-2 Medium model on WikiText-103, the perplexity increases only by 0.04. These results indicate that the proposed method introduces negligible degradation in accuracy during end-to-end inference.

B. Hardware Implementation and Synthesis Results

As outlined in the configuration in Section IV-B, the prototype FRV-SFU-VLIW core adopts an RFU array of size $N = 256$ in the Vector module. Meanwhile, the SFU module is equipped with 64 L-SFUs to cooperate with fully reconfigured RFU resources for vector special-function execution. To sustain this massive compute bandwidth, the internal storage is organized into a VGPR file and an SGPR file. To strictly match the datapath, each VGPR row spans 256×16 bits, while each SGPR entry is 16 bits wide. Both register files are physically realized using four interleaved dual-port SRAM banks ($B = 4$) generated via a commercial memory compiler. The multi-bank organization provides concurrent operand accesses and reduces bank conflicts, while a deeper register file allows more vector states to remain in flight, thereby increasing vector-level parallelism and VLIW issue-slot utilization. Balancing these performance benefits against area overheads, we set both the VGPR and SGPR depths to 256 entries.

TABLE IV
END-TO-END MODEL ACCURACY: FP32 BASELINE VERSUS THE PROPOSED FRV-SFU-VLIW ARCHITECTURE

Method	Format	BERT on GLUE Accuracy (%)			GPT-2 Medium WikiText-103	ViT Classification Accuracy (%)					
		SST-2	QQP	MRPC		DeiT-T	DeiT-S	DeiT-B	Swin-T	Swin-S	Swin-B
Baseline	FP32	92.43	90.33	85.54	16.75	72.17	79.86	81.98	80.61	82.79	85.14
FRV-SFU-VLIW	MIX	92.43 (0.00)	90.35 (+0.02)	85.54 (0.00)	16.79 (+0.04)	72.19 (+0.02)	79.86 (0.00)	81.98 (0.00)	80.61 (0.00)	82.78 (-0.01)	85.16 (+0.02)

Note: MIX denotes the mixed-precision configuration that combines FP16 (E5M10), BF16 (E8M7), and the custom FP19 (E5M13) formats.

TABLE V
MODULE AREA AND POWER BREAKDOWN FOR FRV-SFU-VLIW

Module	Area (mm ²)	Area (%)	Power (W)	Power (%)
VECTOR	1.331	33.1	0.791	21.9
SFU	0.475	11.8	0.300	8.3
SCALAR	0.004	0.1	0.003	0.1
VGPR (SRAM)	2.031	50.5	2.345	65.0
SGPR (SRAM)	0.012	0.3	0.015	0.4
Decoder/Crossbar/Others	0.166	4.2	0.156	4.3
Total (FRV-SFU-VLIW)	4.019	100.0	3.610	100.0

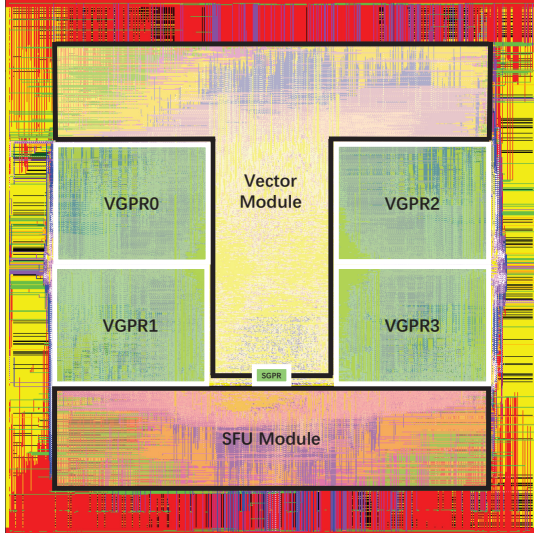


Fig. 5. Physical layout of the proposed FRV-SFU-VLIW prototype.

We implemented the RTL design of the FRV-SFU-VLIW core and synthesized it using a TSMC 28 nm standard-cell library. All floating-point units are custom synthesizable RTL modules. The design successfully achieves a target clock frequency of 1 GHz. Evaluated under the worst-case RC extraction corner, the total core area is 4.019 mm², with an estimated power consumption of 3.61 W. Table V details the comprehensive module-wise area and power breakdown. The physical layout of the FRV-SFU-VLIW core is illustrated in Figure 5. The major blocks, including the SFU module, Vector module, and four VGPR banks, are explicitly annotated in the floorplan.

C. Comparison with State-of-the-Art Accelerators

To comprehensively evaluate the proposed architecture, we benchmark it against three distinct classes of platforms: an ASIC Transformer accelerator, state-of-the-art FPGA-based

nonlinear engines, and a high-end commercial GPU. Since FRV-SFU-VLIW supports a broad range of nonlinear functions expressible through its primitives, exhaustively benchmarking every valid function composition is impractical. We therefore select Softmax, LayerNorm, and GELU as the primary evaluation functions because they are widely used, exhibit clearly distinct computational and dependency characteristics, and, most importantly, are commonly supported by existing accelerators, enabling meaningful cross-platform comparisons.

First, we perform an ASIC-level comparison against the Nonlinear Engine of the Transformer accelerator SwiftTron [16]. Existing works on general-purpose nonlinear acceleration, such as NPE [17] and TATAA [14], are implemented on FPGAs, for which silicon area and power are not directly comparable with our ASIC implementation. We therefore use SwiftTron, a highly optimized INT32-based ASIC for Transformer workloads, as an ASIC reference. Its synthesis and latency breakdowns for nonlinear modules enable a quantitative reference comparison.

Table VI summarizes the synthesis results, cycle efficiency, and energy per cycle of FRV-SFU-VLIW and the SwiftTron nonlinear engine. Following SwiftTron’s evaluation methodology, we report the steady-state average cycles per vector for Softmax and LayerNorm with a sequence length of 256. Compared with SwiftTron under the respective numerical formats and implementations, FRV-SFU-VLIW reduces the cycles per vector by 34.7 $\times$ and 35.2 $\times$ for Softmax and LayerNorm, respectively. Under the technology-normalization model for physical metrics [38], it also requires 5.7 $\times$ smaller area and achieves 2.05 $\times$ lower energy per cycle. The cycle-count differences reflect the combined effects of the respective implementations, including the dedicated reduction datapaths, overlapped VLIW execution, and vector/SFU organization of FRV-SFU-VLIW, while the incremental contribution of FUV is isolated separately in Section VI-D. Beyond these efficiency results, FRV-SFU-VLIW also retains programmability across nonlinear functions expressible through its supported primitives.

Second, Table VII provides an indicative cross-platform comparison with the FPGA-based nonlinear accelerators NPE [17] and TATAA [14]. Since Cyc/Elem is available for all three implementations, we use it to compare their reported execution efficiency under BERT-Base, following the setup in [17]. FRV-SFU-VLIW achieves 5.7 $\times$, 8.6 $\times$, and 2.0 $\times$ lower Cyc/Elem than NPE for Softmax, LayerNorm, and GELU, respectively, and 18.1 $\times$, 18.2 $\times$, and 25.0 $\times$ lower Cyc/Elem than TATAA. These results provide an indicative cycle-level comparison across heterogeneous FPGA and ASIC implementations.

Third, we benchmark standalone Softmax, LayerNorm, and GELU kernels on an NVIDIA RTX 4090 using special-

TABLE VI
SYNTHESIS, CYCLE-EFFICIENCY, AND ENERGY-PER-CYCLE COMPARISON WITH THE ASIC TRANSFORMER ACCELERATOR SWIFTTRON [16]

Work	Tech.	Data Format	Workload Support	Freq. (MHz)	Area (mm ²)	Power (W)	Energy/Cyc. (nJ)	Softmax Cyc./Vec.	LayerNorm Cyc./Vec.
SwiftTron [16] Nonlinear Engine	65 nm	INT32	Softmax, LayerNorm, GELU	143	122.85	7.03	49.16	256	768
SwiftTron [16] Nonlinear Engine (Tech. Normalized)	28 nm	INT32	Softmax, LayerNorm, GELU	331.96	22.80	2.45	7.39	256	768
FRV-SFU-VLIW	28 nm	MIX	Functions expressible through the supported primitives (Softmax, LayerNorm, GELU, Tanh, Sigmoid, etc.)	1000	4.02 (5.7 $\times$)	3.61	3.61 (2.05 $\times$)	7.38 (34.7 $\times$)	21.82 (35.2 $\times$)

Note: (1) Cyc./Vec. is measured for the BERT-Base model with a sequence length of 256 and denotes the steady-state amortized completion interval per vector, rather than the end-to-end latency of an isolated vector. (2) MIX denotes the mixed-precision configuration that combines FP16 (E5M10), BF16 (E8M7), and the custom FP19 (E5M13) formats. (3) The logic-dominated baseline is normalized from 65 nm and 1.0 V to 28 nm and 0.9 V using $s = T_{\text{src}}/T_{\text{tgt}}$ and $v = V_{\text{src}}/V_{\text{tgt}}$, with $f_{\text{tgt}} = f_{\text{src}}s$, $A_{\text{tgt}} = A_{\text{src}}/s^2$, and $P_{\text{tgt}} = P_{\text{src}}/(v^2s)$ [38].

TABLE VII
INDICATIVE CYCLE/ELEMENT COMPARISON UNDER BERT-BASE WITH SEQUENCE LENGTH 512.

Work	Softmax (512 $\times$ 512)	LayerNorm (512 $\times$ 768)	GELU (512 $\times$ 3072)
NPE-2048 [17]	0.1563	0.2421	0.0313
TATAA [14]	0.5000	0.5100	0.3900
FRV-SFU-VLIW	0.0276	0.0281	0.0156
Cyc./Elem. ratio vs [17] / [14] 5.7 $\times$ /18.1 $\times$ 8.6 $\times$ /18.2 $\times$ 2.0 $\times$ /25.0 $\times$			

Note: NPE, TATAA, and FRV-SFU-VLIW operate at 200 MHz, 225 MHz, and 1 GHz, respectively. TATAA shares its arithmetic resources between linear and nonlinear computation.

ized fused CUDA C++ implementations. The Softmax and LayerNorm kernels use NVIDIA CUB’s vendor-optimized `cub::BlockReduce` primitive for block-level reductions: each thread accumulates partial results in registers, followed by warp-/shared-memory-based reductions without materializing intermediate reduction results in global memory.

For each workload, the saturation batch size is selected through a batch-size sweep to operate in the stable large-working-set regime, and the workload is captured into a CUDA Graph. After 50 warm-up replays, 100 graph replays are timed using CUDA events. The median per-instance latency over five runs is reported. During power measurement, the graph is replayed continuously for 30s, while power is sampled using the NVML-backed `nvidia-smi dmon` utility; the mean steady-state power over three runs is reported. Nsight Compute profiling shows that all evaluated GPU workloads sustain 87.7%–93.9% of peak sustained DRAM throughput, confirming high bandwidth utilization.

For a consistent device-resident comparison, GPU timing excludes PCIe transfers but includes device-memory accesses. For FRV-SFU-VLIW, external-memory latency is conservatively modeled using an effective bandwidth of 256 Bytes/cycle per transfer direction, 70% bandwidth utilization, and a 64-cycle setup latency. With Ping-Pong buffering, the steady-state interval is $T_{\text{total}} = \max(T_{\text{compute}}, T_{\text{mem}})$. The modeled memory time remains below the compute time for all evaluated workloads and therefore does not change the reported steady-state execution time.

As shown in Table VIII, the RTX 4090 achieves lower execution latency, as expected from its substantially larger parallel resources and operating frequency of up to 2.65 GHz. Notably, FRV-SFU-VLIW achieves 15.9 $\times$ –26.3 $\times$ higher energy efficiency across all evaluated workloads. This improve-

ment is enabled by the hardware–software co-designed VLIW architecture, which combines programmable function decomposition and static scheduling with heterogeneous scalar, vector, reduction, and special-function datapaths to support multiple nonlinear functions on a unified programmable substrate. The results show that FRV-SFU-VLIW effectively retains software programmability while delivering the substantial efficiency benefits of domain-specific acceleration.

D. Quantitative Analysis of the FUV Enhancement

To isolate the incremental benefit of FUV, we compare two matched FRV-SFU-VLIW implementations with FUV disabled and enabled. Both use the same process, frequency, vector width, register organization, and reduction datapaths. The FUV-disabled implementation contains one scalar S-SFU, 32 fixed vector SFUs with local quadratic-computation datapaths, and 256 non-reconfigurable baseline arithmetic units, whereas the FUV-enabled implementation uses lightweight L-SFUs and reconfigurable RFUs. The FUV-enabled implementation introduces only 1.6% area overhead and 0.8% power overhead.

Table IX further reports the resource-normalized efficiency of the two matched implementations. For SFU-bound workloads, FUV improves both throughput per area and throughput per watt. In particular, GELU achieves a 2.00 $\times$ speedup, reaching the theoretical upper bound of the current prototype configuration, as FUV doubles the maximum effective vector-SFU parallelism from 32 in the disabled configuration to 64 in the enabled configuration. The gains for Softmax, LogSoftmax, Softplus, and LogSumExp are more moderate because data dependencies among DAG nodes limit the exploitable parallelism, whereas the LayerNorm cycle remains unchanged because vector special-function execution is not its bottleneck. These results confirm that FUV is a low-overhead enhancement mechanism for alleviating workload-dependent resource imbalance and contributes to, rather than solely determines, the cross-platform advantages of the complete FRV-SFU-VLIW architecture.

To further examine how the compiler exploits this flexibility under different workload scales, we compare the scheduling cycles of the FUV-disabled and FUV-enabled implementations for Softmax (32 $\times$ Y), where Y denotes the vector length. Figure 6 shows a clear workload-dependent mode-selection behavior. For very small vectors, the benefit is limited because fixed pipeline and dependency overheads account for a larger fraction of the total cycles, and the additional vector-SFU parallelism does

TABLE VIII
AMORTIZED DEVICE EXECUTION TIME AND ENERGY EFFICIENCY COMPARISON AGAINST OPTIMIZED STANDALONE CUDA NONLINEAR KERNELS

Seq. Length	Workload (Shape)	Amortized Time (μ s)		Energy Efficiency (G-Elems/s/W)		
		RTX 4090	FRV-SFU-VLIW	RTX 4090	FRV-SFU-VLIW	Gain
256	Softmax (256×256)	0.28	1.89	0.52	9.61	18.6 $\times$
	LayerNorm (256×768)	0.85	5.59	0.53	9.75	18.5 $\times$
	GELU (256×3072)	3.41	12.32	0.67	17.69	26.3 $\times$
512	Softmax (512×512)	1.14	7.23	0.51	10.04	19.5 $\times$
	LayerNorm (512×768)	1.70	11.05	0.53	9.86	18.8 $\times$
	GELU (512×3072)	6.82	24.60	0.68	17.71	26.2 $\times$
1024	Softmax (1024×1024)	4.54	28.55	0.51	10.18	19.8 $\times$
	LayerNorm (1024×768)	3.41	22.08	0.62	9.84	15.9 $\times$
	GELU (1024×3072)	13.65	49.18	0.68	17.72	26.2 $\times$

Note: Results are standalone-kernel measurements rather than end-to-end LLM or attention latency. The shapes are derived from representative BERT-Base dimensions: hidden size 768, intermediate size 3072, and an $[S, S]$ attention-score matrix. GPU kernels use FP16 inputs/outputs with FP32 accumulation. The saturation batch sizes for sequence lengths 256, 512, and 1024 are 2048/512/128 for Softmax, 682/341/170 for LayerNorm, and 170/85/42 for GELU. GPU time denotes amortized per-instance device execution time under saturated CUDA Graph replay, excluding host-device transfers.

TABLE IX
MATCHED RESOURCE AND PERFORMANCE COMPARISON WITH FUV DISABLED/ENABLED (D/E).

Workload	Cycle (D/E)	BFU/RFU Util. (% D/E)	Fixed-SFU/L-SFU Util. (% D/E)	Thr./Area (GElem/s/mm ² , D/E)	Thr./W (GElem/s/W, D/E)	Energy/Elem (pJ, D/E)	Speedup
Softmax (512×512)	8292/7228	24.7/85.0	98.8/56.7	7.99/9.02	8.83/10.05	113.24/99.54	1.15 $\times$
GELU (512×3072)	49179/24603	0.0/99.9	99.9/99.9	8.08/15.90	8.93/17.71	111.94/56.47	2.00 $\times$
LayerNorm (512×768)	11048/11048	69.5/69.5	0.0/0.0	9.00/8.85	9.94/9.86	100.59/101.43	1.00 $\times$
LogSoftmax (512×2048)	33167/28268	24.7/93.2	98.8/59.4	7.99/9.23	8.83/10.28	113.24/97.32	1.17 $\times$
Softplus (128×1024)	8262/5344	6.2/86.2	99.2/76.7	4.01/6.10	4.43/6.79	225.66/147.19	1.55 $\times$
LogSumExp (256×1024)	8311/6255	12.3/81.9	98.6/65.5	7.97/10.43	8.81/11.61	113.50/86.14	1.33 $\times$

Note: Metrics are calculated at 1 GHz using the workload element count. The D/E implementation areas and powers are 3.956/4.02 mm² and 3.58/3.61 W, respectively.

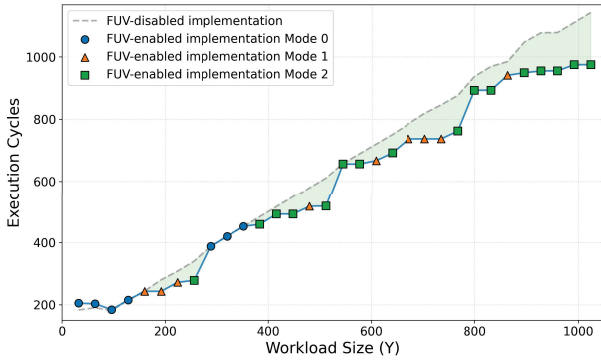


Fig. 6. Scheduling-cycle comparison between the matched FUV-disabled and FUV-enabled implementations for Softmax ($32 \times Y$), where Y denotes the vector length. The markers indicate the FUV mode selected by the proposed static multi-mode scheduler for the FUV-enabled implementation.

not always offset the reduced ordinary-vector resources. Across different values of Y , the scheduler selects different FUV modes according to the workload-dependent balance between vector arithmetic and special-function demands, with a more evident cycle advantage at larger scales. Quantitatively, the FUV-enabled implementation reduces the cycle count from 340 to 278 at $Y = 256$ and from 1142 to 975 at $Y = 1024$, with the maximum reduction reaching 18.2%. These results show that the proposed static multi-mode scheduling strategy can effectively translate the flexibility of FUV into cycle-level performance improvement across different workload scales.

E. Discussion

Although Table V shows that the VGPR is the largest physical area and power component, this does not translate into a runtime register-bandwidth bottleneck. We further analyze the data-movement pressure and dominant bottlenecks for the three representative 512-length workloads listed in Table IX. The VGPR read/write bank-port utilizations are 21.3%/14.2% for Softmax, 12.5%/12.5% for GELU, and 35.3%/21.2% for LayerNorm. These results show that, with the deep, multi-banked VGPR and dedicated reduction datapaths, register traffic is not the dominant bottleneck under the evaluated configurations. Under the same HBM model as Section VI-C, the corresponding modeled memory-service ratios $T_{\text{mem}}/T_{\text{comp}}$ are 41.2%, 71.6%, and 40.4%, respectively, which indicates that external transfers are hidden by Ping-Pong execution in steady state. Under the evaluated configurations, the FRV-SFU-VLIW execution intervals therefore remain compute-bound. The dominant compute bottleneck is workload-dependent: GELU is constrained by the maximum vector-SFU throughput, LayerNorm by inter-node dependencies and long-latency critical-path operations, and Softmax by both factors.

VII. CONCLUSION

This paper presents FRV-SFU-VLIW, a fully reconfigurable and highly programmable VLIW architecture for nonlinear function acceleration. By integrating a lightweight SFU organization, an RFU-based Vector module, the FUV mechanism, and a Static Multi-Mode Scheduler, FRV-SFU-VLIW provides a flexible hardware-software substrate for diverse and evolving

nonlinear workloads expressible through the supported primitives. Implemented with TSMC 28 nm CMOS technology, FRV-SFU-VLIW achieves up to $35.2\times$ fewer cycles per vector and a $5.7\times$ reduction in technology-normalized area over SwiftTron nonlinear modules, $2.0\times$ – $25.0\times$ fewer cycles per element than FPGA-based general nonlinear accelerators in an indicative cross-platform comparison, and $15.9\times$ – $26.3\times$ energy-efficiency gains over an NVIDIA RTX 4090 GPU for standalone nonlinear functions, while maintaining negligible end-to-end accuracy loss. Together, the demonstrated programmability, extensibility, execution efficiency, and numerical accuracy position FRV-SFU-VLIW as a promising substrate for nonlinear computation demands in future AI and related computing applications.

ACKNOWLEDGMENTS

This work was supported by ZTE Industry-University-Institute Cooperation Funds under Grant No. IA20241120003-PO0001.

REFERENCES

- [1] J. Wu, "Introduction to convolutional neural networks," *National Key Lab for Novel Software Technology. Nanjing University. China*, vol. 5, no. 23, p. 495, 2017.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [3] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.
- [4] A. F. Agarap, "Deep learning using rectified linear units (relu)," *arXiv preprint arXiv:1803.08375*, 2018.
- [5] P. Ramachandran, B. Zoph, and Q. V. Le, "Swish: a self-gated activation function," *arXiv preprint arXiv:1710.05941*, vol. 7, no. 1, p. 5, 2017.
- [6] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softmax: Hardware/software co-design of an efficient softmax for transformers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 469–474, IEEE, 2021.
- [7] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [8] Y. Qiao, F. Yang, Y. Zhang, X. Xiong, X. Yao, and H. Yao, "Fsmm: An efficient matrix multiplication accelerator supporting flexible sparsity," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–9, 2024.
- [9] M. Imani, S. Gupta, Y. Kim, and T. Rosing, "Floatpim: In-memory acceleration of deep neural network training with high precision," in *Proceedings of the 46th International Symposium on Computer Architecture*, pp. 802–815, 2019.
- [10] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 44, no. 3, pp. 367–379, 2016.
- [11] S. Krishnaprasad, "Uses and abuses of amdahl's law," *Journal of Computing Sciences in colleges*, vol. 17, no. 2, pp. 288–293, 2001.
- [12] V. Potocnik, L. Colagrande, T. Fischer, L. Bertaccini, D. J. Pagliari, A. Burrello, and L. Benini, "Optimizing foundation model inference on a many-tiny-core open-source risc-v platform," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024.
- [13] T. Peng, T. Xia, J. Qin, and S. Q. Zhang, "Haan: A holistic approach for accelerating normalization operations in large language models," in *2025 Design, Automation Test in Europe Conference (DATE)*, pp. 1–7, 2025.
- [14] J. Wu, M. Song, J. Zhao, Y. Gao, J. Li, and H. K.-H. So, "Tataa: Programmable mixed-precision transformer acceleration with a transformable arithmetic architecture," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 1, pp. 1–31, 2025.
- [15] W. Li, D. Lyu, G. Wang, A. Hu, N. Xu, and G. He, "Hardware-oriented algorithms for softmax and layer normalization of large language models," *Science China Information Sciences*, vol. 67, no. 10, p. 200404, 2024.
- [16] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, "Swifttron: An efficient hardware accelerator for quantized transformers," in *2023 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–9, IEEE, 2023.
- [17] H. Khan, A. Khan, Z. Khan, L. B. Huang, K. Wang, and L. He, "Npe: An fpga-based overlay processor for natural language processing," *arXiv preprint arXiv:2104.06535*, 2021.
- [18] Y. LIU, R. GAO, C. ZENG, and Y. LIU, "A transformer-based end-to-end receiver design for wi-fi 7 physical layer," *ZTE COMMUNICATIONS*, vol. 23, no. 4, 2025.
- [19] Y. Lang, H. Chen, W. Yanfeng, L. Ronghua, J. Feng, and G. Ying, "Video enhancement network based on cnn and transformer," *ZTE COMMUNICATIONS*, vol. 22, no. 4, 2024.
- [20] F. Zhu, X. Wang, C. Zhu, and C. Huang, "Liquid neural networks: Next-generation ai for telecom from first principles," *ZTE COMMUNICATIONS*, vol. 23, no. 2, 2025.
- [21] M. T. Heath, *Scientific computing: an introductory survey, revised second edition*. SIAM, 2018.
- [22] W. Shen, J. Jiang, M. Li, and S. Liu, "Efficient cordic-based activation functions for rnn acceleration on fpgas," *IEEE Transactions on Artificial Intelligence*, 2024.
- [23] V. Shatravin, D. Shashev, and S. Shidlovskiy, "Sigmoid activation implementation for neural networks hardware accelerators based on reconfigurable computing environments for low-power intelligent systems," *Applied Sciences*, vol. 12, no. 10, p. 5216, 2022.
- [24] L. Li, S. Zhang, and J. Wu, "An efficient hardware architecture for activation function in deep learning processor," in *2018 IEEE 3rd International Conference on Image, Vision and Computing (ICIVC)*, pp. 911–918, IEEE, 2018.
- [25] C.-H. Chang, H.-Y. Kao, and S.-H. Huang, "Hardware implementation for multiple activation functions," in *2019 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW)*, pp. 1–2, IEEE, 2019.
- [26] C. Zhengbo, T. Lei, and C. Zuoning, "Research and design of activation function hardware implementation methods," in *Journal of Physics: Conference Series*, vol. 1684, p. 012111, IOP Publishing, 2020.
- [27] R. Andri, E. Reggiani, and L. Cavigelli, "Flex-sfu: Activation function acceleration with non-uniform piecewise approximation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [28] K. Hu, J. Xie, K. Han, and F. Yang, "Highly efficient module design for general multivariate nonlinear functions using stochastic computing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2026.
- [29] K. Qian, Y. Liu, Z. Zhang, and K. Wang, "Efficient implementation of activation function on fpga for accelerating neural networks," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2023.
- [30] X. Yang and T. Su, "Efa-trans: An efficient and flexible acceleration architecture for transformers," *Electronics*, vol. 11, no. 21, p. 3550, 2022.
- [31] W. Wang, S. Zhou, W. Sun, P. Sun, and Y. Liu, "Sole: Hardware-software co-design of softmax and layernorm for efficient transformer inference," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2023.
- [32] D. Han and A. P. Chandrakasan, "Mega. mini: A universal generative ai processor with a new big/little core architecture for npu," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68, pp. 1–3, IEEE, 2025.
- [33] Y. Qin, Y. Wang, J. Wang, Z. Lin, Y. Zhao, S. Wei, Y. Hu, and S. Yin, "23.8 an 88.36 tops/w bit-level-weight-compressed large-language-model accelerator with cluster-aligned int-fp-gemm and bi-dimensional workflow reformulation," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68, pp. 420–422, IEEE, 2025.
- [34] R. Kim, D. Lee, J. Kim, J. Park, and S. E. Lee, "Hardware accelerator for approximation-based softmax and layer normalization in transformers," *Electronics*, vol. 14, no. 12, p. 2337, 2025.
- [35] S. Ghodrati, S. Kinzer, H. Xu, R. Mahapatra, Y. Kim, B. H. Ahn, D. K. Wang, L. Karthikeyan, A. Yazdanbakhsh, J. Park, et al., "Tandem processor: Grappling with emerging operators in neural networks," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 1165–1182, 2024.
- [36] S. Savas, Y. Atwa, T. Nordström, and Z. Ul-Abdin, "Using harmonized parabolic synthesis to implement a single-precision floating-point square root unit," in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 621–626, IEEE, 2019.
- [37] R. Pachón and L. N. Trefethen, "Barycentric-remez algorithms for best polynomial approximation in the chebfun system," *BIT Numerical Mathematics*, vol. 49, no. 4, pp. 721–741, 2009.
- [38] U. Lee and M. H. Sunwoo, "Hardware implementation of fast polar decoders based on a simplified control unit," *Electronics Letters*, vol. 61, no. 1, p. e70132, 2025.



Ke Hu received B.S. degree from Fudan University in 2022. She is pursuing her PhD degree at Fudan University in Shanghai, China. Her research interests include sparse matrix processing, LU decomposition, Stochastic Computing, and nonlinear function calculation in neural networks.



Lei Xu is an FPGA Development Engineer of ZTE Corporation. His main research directions include artificial intelligence, microarchitecture, FPGA design, etc.



Zikai Zhao received the B.S. degree from Fudan University in 2025. He is pursuing the master degree with the State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai, China. His research interests include AI chips, nonlinear function calculation in neural networks.



Wenhao Cai is a Technical Pre-research Engineer of ZTE Corporation. His main research directions include deep learning algorithms, large language models, simulation computing, wireless communication systems, etc.



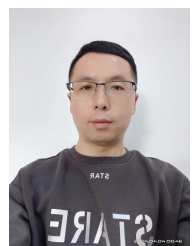
Yuxuan Qiao received the B.S. degree from Huazhong University of Science and Technology, Wuhan, China, in 2022. He is pursuing the master degree with the State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai, China. His research interests include Sparse matrix multiplication accelerator, AI chips, reliable matrix multiplication accelerator.



Xiankui Xiong is the Chief Architect of Wireless at ZTE Corporation and Head of the Forward-looking Group of the "Intelligent Computing" Technology Committee. He has long been engaged in research on computing systems and architectures, advanced computing paradigms, and heterogeneous computing accelerators.



Yifan Song received B.S. degree from Fudan University in 2023. She is currently pursuing the Ph.D. degree with the State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai, China. Her research interests include sparse matrix multiplication accelerator, LLM accelerator design, and reliable AI accelerator design.



Jun Liu is a Senior Wireless Intelligent Computing Expert of ZTE Corporation. His main research directions include deep learning algorithms, AI compilers, AI accelerator architecture design and simulation, etc.



Fan Yang received a B.S. degree from Xi'an Jiaotong University in 2003 and a Ph.D. degree from Fudan University, Shanghai, China, in 2008.

He is currently a full professor in the Microelectronics Department at Fudan University. His research interests include model order reduction, circuit simulation, high-level synthesis, acceleration of artificial neural networks, yield analysis, and design for manufacturability.